Efficient LLM Serving in Resource-Constrained Environments: An Empirical Validation of PagedAttention

Nicholas Lisle
Student ID: 5815578
Class: CAP6441_CMB
ni611897@ucf.edu

Abstract

The rapid scaling of Large Language Models (LLMs) has shifted the primary deployment bottleneck from pure computational constraints to memory management, specifically regarding the Key-Value (KV) cache during autoregressive inference. Traditional serving frameworks allocate contiguous memory blocks for the KV cache, leading to severe internal and external fragmentation that heavily degrades throughput and limits concurrent user capacity. This paper empirically evaluates PagedAttention, a memory management algorithm implemented within the vLLM framework, by validating its efficacy in a resource-constrained edge environment. Using a Tesla T4 GPU (16GB VRAM) and the TinyLlama-1.1B model, we conduct a head-to-head performance analysis against the standard HuggingFace `generate()` pipeline. Our results demonstrate that PagedAttention achieves a 2.61x increase in token throughput (425.28 tokens/sec vs. 162.86 tokens/sec) and a 61.7% reduction in end-to-end latency. Furthermore, we analyze the memory pre-allocation strategy of vLLM and show that systems-level virtual memory mapping effectively eliminates the reservation waste inherent to contiguous allocation. These findings confirm that advanced memory paging techniques are critical not only for enterprise-scale A100 clusters but also for democratizing efficient LLM deployment on consumer-grade hardware.

1 Introduction

The advent of transformer-based Large Language Models (LLMs) has catalyzed breakthroughs across natural language understanding, generation, and complex reasoning tasks. However, transitioning these models from research environments to production serving systems presents substantial engineering challenges. While model training is bound by floating-point operations per second (FLOPs), inference is overwhelmingly memory-bound. Specifically, the autoregressive nature of text generation requires the system to store the attention keys and values for all previously generated tokens to avoid redundant recomputation. This mechanism, known as the KV cache, scales linearly with both sequence length and batch size, rapidly exhausting GPU Video RAM (VRAM) long before compute units are fully saturated.

Standard inference libraries (such as the HuggingFace Transformers library) manage the KV cache naively by allocating contiguous blocks of memory based on the maximum potential sequence length of a request. Because the actual generated sequence length is fundamentally unpredictable, this approach results in massive memory waste. It is estimated that traditional systems waste between 60% and 80% of available VRAM due to internal fragmentation, external fragmentation, and reservation waste.

To address this, Kwon et al. introduced PagedAttention and the vLLM framework, which adapts traditional operating system virtual memory principles to GPU memory management. By dividing the KV cache into discrete, fixed-size blocks dynamically mapped via a block table, vLLM drastically reduces fragmentation. While the original literature validates this methodology on massive,

multi-GPU clusters using parameter-heavy models, there remains a critical need to understand how these systems scale down.

In this paper, we expand upon the foundational PagedAttention research by validating its efficiency in a resource-constrained environment. We deploy a 1.1-billion-parameter model on a single Tesla T4 GPU, demonstrating that systems-level memory optimization yields profound performance benefits—specifically, a 2.61x throughput multiplier—even on edge-tier and consumer hardware.

2 Related Work

Autoregressive Inference and the KV Cache: The standard implementation of transformer inference relies heavily on caching previous states. Wolf et al. documented the foundational HuggingFace Transformers pipeline, which prioritized ease of use and model accessibility over high-concurrency throughput. While effective for single-batch experimentation, its reliance on contiguous memory tensors severely limits production scalability.

Algorithmic Acceleration Techniques: To combat inference latency, several algorithmic interventions have been proposed. Dao introduced FlashAttention and FlashAttention-2, which optimize hardware memory reads/writes (IO-awareness) by fusing attention operations and tiling data in SRAM. While FlashAttention drastically improves the speed of the attention computation, it does not address the overarching storage capacity issue in the KV cache for long sequences. Similarly, Speculative Decoding (Leviathan et al.) reduces latency by using a smaller draft model to predict tokens, which are then verified in parallel by a larger target model. However, speculative methods require secondary models and complex orchestration, and they still demand robust underlying memory management.

Systems-Level Memory Management: Kwon et al. bridged the gap between OS-level architecture and AI serving with vLLM. By formalizing PagedAttention, they provided a framework that uncouples physical memory adjacency from logical sequence continuity. This paper builds directly upon Kwon et al.'s work by shifting the testing paradigm from highly parallelized H100/A100 data centers to constrained environments, demonstrating the universal applicability of memory paging in AI.

3 Method

3.1 The KV Cache Memory Bottleneck

During autoregressive generation, a transformer model generates a sequence token by token. To compute the attention scores for a new token, the model must attend to all previous tokens. If the system were to recompute these states at every step, the time complexity would scale quadratically, rendering real-time generation impossible.

The KV cache mitigates this by storing the Key (K) and Value (V) vectors for each layer. The memory footprint of the KV cache is dictated by the formula:

$$Memory = 2 \times B \times S \times L \times H \times P \tag{1}$$

where B is the batch size, S is the sequence length, L is the number of layers, H is the hidden dimension size, and P is the precision byte size (e.g., 2 bytes for FP16). As S grows unpredictably during generation, the memory requirement dynamic becomes highly volatile.

3.2 The Flaws of Contiguous Allocation

In baseline frameworks like HuggingFace, memory allocators require tensors to occupy contiguous addresses in VRAM. To avoid constant reallocation overhead, the system pre-allocates a block equivalent to the maximum supported context window (e.g., 2048 tokens) the moment a request enters the queue. This leads to three distinct forms of memory waste:

- **Reservation Waste:** Memory is held hostage for future tokens that have not yet been generated.
- **External Fragmentation:** Different requests finish at different times, leaving scattered "holes" of free memory across the GPU that are too small to house new contiguous allocations.
- **Internal Fragmentation (The 4-Token Paradigm):** Consider an allocation block reserved for a maximum sequence length of 2048 tokens. If a user prompts the model and it generates a brief answer terminating after only 4 tokens, the remaining 2044 slots in that block remain locked. In this scenario, 99.8% of the allocated memory sits entirely empty but inaccessible to other concurrent users.

Collectively, these inefficiencies result in up to 80% of VRAM being wasted.

3.3 PagedAttention and vLLM

PagedAttention solves the contiguous allocation constraint by applying virtual memory paging. The KV cache is divided into fixed-size physical blocks (pages), each containing the keys and values for a fixed number of tokens.

A central "Block Table" maps the logical progression of a user's prompt to these physical pages, meaning the pages can be stored non-contiguously anywhere in VRAM. When a user generates a new token, the system simply writes to the current active block. If the block fills up, vLLM dynamically requests a single new block from the operating pool. Consequently, internal fragmentation is reduced to only the unused slots in the final, partially filled block, resulting in total memory waste of under 4%. Furthermore, PagedAttention enables Continuous Batching, allowing new requests to be seamlessly interleaved into the GPU execution cycle at the exact millisecond when previous requests terminate, maximizing hardware saturation.

4 Experiments

4.1 Experimental Setup

To validate the methodology under strict hardware limits, we established a reproducible benchmark utilizing a Google Colab instance equipped with a single NVIDIA Tesla T4 GPU (16GB VRAM, Compute Capability 7.5).

Model Configuration: We utilized `TinyLlama/TinyLlama-1.1B-Chat-v1.0`. This 1.1-billion-parameter architecture relies on standard LLaMA mechanics but fits comfortably within consumer-grade VRAM limits, making it ideal for isolating the system-level memory management differences without triggering irrelevant architectural bottlenecks. The weights were loaded in FP16 precision.

Workload Simulation: We simulated a high-concurrency production environment by passing a static batch ($B = 8$) of diverse prompts simultaneously. The maximum generation length was capped at 100 new tokens per prompt.

Baselines: 1. **Standard Inference:** HuggingFace `AutoModelForCausalLM` using standard contiguous KV-caching. 2. **Optimized Inference:** vLLM engine utilizing PagedAttention. Due to the T4's constrained VRAM, vLLM's `gpu_memory_utilization` parameter was restricted to 0.4 (40%) to create a controlled paging pool without inducing OS-level out-of-memory kernel panics.

4.2 Evaluation Metrics

The primary metrics collected were:

- **Throughput:** Measured in tokens per second, calculated as the sum of all newly generated tokens across the batch divided by total execution time.

- **Latency:** End-to-end execution time for the entire batch to complete generation.
- **Peak VRAM:** The maximum GPU memory allocated during the execution phase, tracked via `torch.cuda.max_memory_allocated()`.

5 Results and Analysis

The empirical data collected from our execution runs highlight a stark performance disparity between standard contiguous memory management and paged virtual memory management.

5.1 Throughput and Latency Improvements

As detailed in the results, the vLLM engine achieved a massive performance gain over the baseline.

Table 1: Performance Comparison: HuggingFace vs. vLLM

Metric	HuggingFace (Standard)	vLLM (PagedAttention)	Improvement
Throughput	162.86 tokens/sec	**425.28 tokens/sec**	**2.61x Faster**
End-to-End Latency	4.91 seconds	**1.88 seconds**	**61.7% Reduction**
Peak Execution VRAM	2.08 GB	5.92 GB (Pool)	System Stability

The HuggingFace implementation processed the batch of 8 prompts in 4.91 seconds, resulting in a throughput of 162.86 tokens/sec. In contrast, the vLLM implementation completed the identical workload in just 1.88 seconds, yielding 425.28 tokens/sec. This 2.61x multiplier confirms that the bottleneck in standard serving is not the FLOP capacity of the Turing architecture T4 GPU, but rather the memory IO and allocation stalling. vLLM's Continuous Batching allows the GPU cores to remain actively computing matrix multiplications, unhindered by contiguous memory reallocation delays.

5.2 Memory Dynamics and the Pre-allocation Paradox

A superficial reading of the peak VRAM metrics might suggest that HuggingFace is more "memory efficient," as it recorded a peak execution footprint of 2.08 GB compared to vLLM's 5.92 GB. However, this highlights the fundamental architectural difference between the two systems.

HuggingFace utilizes "lazy" or on-demand allocation. While the tracked peak appears low for this specific short burst, the VRAM's memory layout is highly fragmented. If the batch size were increased or the sequence length were extended to 2048 tokens, external fragmentation would rapidly lead to an Out-Of-Memory (OOM) crash.

Conversely, vLLM employs a strict pre-allocation strategy. Upon initialization, it intentionally reserves a large, dedicated VRAM block (as dictated by the 0.4 utilization parameter) for its Block Pool. The 5.92 GB represents this secure, pre-reserved virtual memory space. During execution, the actual additional memory used was effectively near zero ($\sim$0.01 GB overhead). This paradox proves that vLLM trades an upfront, predictable reservation of memory for the guarantee of zero fragmentation during runtime, ensuring absolute system stability under heavy load.

6 Discussion

6.1 Implications for Resource-Constrained Environments

The original PagedAttention literature focused on high-end datacenter applications. Our results extend the literature by proving that virtual memory management is equally critical for Edge AI and consumer hardware. Devices with 8GB to 16GB of unified memory or VRAM cannot afford the 80% fragmentation waste of standard contiguous serving. By using vLLM, developers can deploy

models such as TinyLlama or LLaMA-3-8B locally and achieve datacenter-like tokenization speeds without hardware upgrades. This effectively democratizes high-throughput local AI, lowering the total cost of ownership (TCO) for local inference applications.

6.2 Limitations and Trade-offs

While the throughput gains are undeniable, PagedAttention is strictly an inference-side optimization. It offers no acceleration for model training, fine-tuning (e.g., LoRA), or gradient updates. Additionally, implementing vLLM introduces deployment complexity. Maintaining the Block Table requires minimal CPU overhead and specific CUDA kernel compatibility. Finally, the pre-allocation strategy means that the GPU running vLLM cannot easily share its VRAM with other concurrent applications, as vLLM "locks" the paged pool upon startup.

7 Conclusion

Efficient deployment of Large Language Models is predicated on mastering memory management. This paper provides an empirical validation of the PagedAttention methodology, demonstrating that treating GPU VRAM similarly to OS virtual memory yields profound efficiency gains. Through our controlled deployment of TinyLlama on a resource-constrained Tesla T4 GPU, we demonstrated that eliminating contiguous memory fragmentation increases token throughput by 2.61x and reduces latency by over 60%. By replacing rigid memory reservation with dynamic, non-contiguous block mapping, systems can fully utilize their hardware capacity. As LLMs continue to scale, bridging the gap between systems engineering and machine learning architecture—as vLLM has successfully done—will remain the most viable path toward sustainable AI deployment.

Disclosure of AI Tools

Grammarly was utilized strictly for grammatical corrections and syntactical refinement during the drafting of this manuscript. No other generative AI assistants were used to produce the content or experimental code of this paper.

References

[1] Dao, T. (2023). *FlashAttention-2: Faster attention with better parallelism and work partitioning*. arXiv preprint arXiv:2307.08691.

[2] Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., ... & Stoica, I. (2023). *Efficient memory management for large language model serving with PagedAttention*. Proceedings of the 29th Symposium on Operating Systems Principles (SOSP '23).

[3] Leviathan, Y., Kalman, M., & Matias, Y. (2023). *Fast inference from transformers via speculative decoding*. Proceedings of the 40th International Conference on Machine Learning (ICML).

[4] Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., ... & Chintala, S. (2019). *PyTorch: An imperative style, high-performance deep learning library*. Advances in Neural Information Processing Systems, 32.

[5] Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., ... & Rush, A. M. (2020). *Transformers: State-of-the-art natural language processing*. Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations.

[6] Zhang, P., Zeng, G., Wang, T., & Lu, W. (2024). *TinyLlama: An open-source small language model*. arXiv preprint arXiv:2401.02385.

[7] Grammarly. (2026). *Grammarly: AI Writing Assistance*. Grammarly Inc.